



University of Science - VNUHCM
FACULTY OF INFORMATION TECHNOLOGY

Final Project Report

Clinic Management System

Course: Fundamental of Programming

Authors:

Huynh Thanh An	(25127178)
Luong Duy Phuoc	(24127509)
Ly Si Vi	(24127587)
Tran Ngoc My Trinh	(25127258)

Instructor:

M.Sc. Tran Duy Quang

Class:

25CA

Ho Chi Minh City, May 2026

Contents

1	Project Overview	2
1.1	Introduction	2
1.2	Objectives	2
1.3	Platform and Build	2
1.4	Repository Structure	2
2	Feature Overview and Learning Outcome Mapping	3
3	System Architecture	5
3.1	Module Structure	5
3.2	Data Flow	5
4	Detailed Feature Implementation	6
4.1	Patient Management - Dynamic Array (G1.2, G1.3)	6
4.2	Binary File Persistence (G2.3)	7
4.3	Search and Sort - Quick Sort via Pointers (G1.4, G2.1)	7
4.4	Examination Queue - FIFO Linked List (G3.2)	8
4.5	Visit History - Singly Linked List (G3.1)	8
4.6	Medicine Storage - LIFO Stack (G3.2)	9
4.7	Statistics - 2D Array (G1.4)	10
4.8	Report Export (G2.3)	10
4.9	Console UI (G4.1)	10
5	Testing	11
6	Team Contributions	12
7	Conclusion	13

1. Project Overview

1.1. Introduction

The **Clinic Management System** is a console-based C++ application designed to automate the core operational workflows of a small clinic. The system provides comprehensive patient record management, an examination queuing mechanism, a medication inventory module, visit-history tracking, statistical reporting, and persistent binary-file storage - all implemented through a modular, multi-file architecture.

The project was developed as the final assignment for the *Fundamental of Programming* course (Class 25CA, 2026), with the explicit goal of applying every data-structure category taught in the curriculum: dynamic arrays, linked lists, queues, stacks, and two-dimensional arrays.

1.2. Objectives

- Build a fully functional C++ console application that manages clinic operations end-to-end.
- Apply fundamental data structures: dynamic arrays, pointers, structs, linked lists, queues, stacks, and 2D arrays.
- Ensure persistent data storage between program sessions via binary files.
- Organise code in a modular, maintainable fashion using separate `.h/.cpp` file pairs.
- Satisfy all learning outcomes G1, G2, G3, and G4 of the course.

1.3. Platform and Build

- **Language:** C++14
- **Platform:** Windows (MinGW/g++) - uses `<windows.h>` and `<conio.h>` for console UI
- **Build:** `cd src && g++ *.cpp -o ../Clinic.exe -std=c++14`
- **Duration:** 8 weeks (12/03/2026 - 05/05/2026)
- **Version Control:** GitHub
- **GitHub Repository:** <https://github.com/htan2612/MediCare-Management-System>
- **YouTube Demo Video:** *(link to be provided)*

1.4. Repository Structure

The repository is organised into three top-level folders to separate concerns clearly:

```
MediCare-Management-System/  
  src/      C++ source and header files (20 files)  
  report/   LaTeX report source + university logo  
  video/    Demo video (link.txt)  
  README.md
```

All C++ source files reside under `src/`. The compiled executable and the `patients.bin` save file are generated in the project root when the build command is run from there.

2. Feature Overview and Learning Outcome Mapping

Table 1 lists all eleven tasks from the project proposal, along with the corresponding data structure, learning outcome, and responsible member.

Table 1: Feature - Learning Outcome - Member Mapping

#	Task	LO	Data Structure	Requirement	Member
1	UI & Navigation	G4.1	Windows API	ASCII menu, arrow-key navigation, loading effects, box-drawing frames	Huynh Thanh An
2	Patient CRUD	G1.2, G1.3	Struct, dyn. array	Add / View / Edit / Delete patient records using dynamic array	Ly Si Vi
3	Binary File Storage	G2.3	Binary file	Read/write binary file to persist patient list	Ly Si Vi
4	Search & Sort	G1.4, G2.1	Pointer, dyn. array	Search by ID/Name; sort list (A-Z, Age) using pointers	Ly Si Vi
5	Examination Queue	G3.2	Linked-list queue	FIFO patient call system using Queue	Luong Duy Phuoc
6	Visit History	G3.1	Singly linked list	Store each patient's visits in a singly linked list	Luong Duy Phuoc
7	Medicine Storage (Stack)	G3.2	Linked-list stack	Manage medicine import/export with LIFO (Stack)	Huynh Thanh An
8	Statistics (2D Array)	G1.4	2D array	Aggregate visit counts by Day $\times$ Month	Tran Ngoc My Trinh
9	Report Export	G2.3	File I/O	Export patient list & statistics to a text file	Tran Ngoc My Trinh
10	Report & Docs	—	—	Write project report, algorithm diagrams, bug fixes	Huynh Thanh An
11	Input Validation	G1.2, G1.3	do-while, if-else	Validate age (1-150), non-empty name, unique ID	Huynh Thanh An

3. System Architecture

3.1. Module Structure

All source code is contained in the `src/` folder and decomposed into eleven modules, each encapsulating a single responsibility.

Table 2: Source File Overview (`src/` folder)

File(s)	LO	Responsibility
<code>src/Patient.h</code>	–	Core structs: <code>Patient</code> , <code>PatientList</code>
<code>src/History.h/cpp</code>	G3.1	Singly linked list for visit history
<code>src/PatientCRUD.h/cpp</code>	G1.2, G1.3	CRUD operations + dynamic array resize
<code>src/PatientSearchSort.h/cpp</code>	G1.4, G2.1	Name/ID search + Quick Sort
<code>src/PatientFile.h/cpp</code>	G2.3	Binary save/load including visit histories
<code>src/Queue.h/cpp</code>	G3.2	FIFO examination queue
<code>src/Medicine.h/cpp</code>	G3.2	LIFO medicine stack + interactive sub-menu
<code>src/Statistics.h/cpp</code>	G1.4	2D visit-count array + date parsing
<code>src/Reports.h/cpp</code>	G2.3	Export timestamped <code>.txt</code> report
<code>src/Clinic.h/cpp</code>	G4.1	Console UI: colours, box drawing, arrow-key menu
<code>src/main.cpp</code>	G4.1	Entry point - wires all modules together

3.2. Data Flow

At startup, `main()` initialises all data structures, displays the boot banner, and loads `patients.bin` (located in the project root) via `PatientFile`. The user navigates the arrow-key main menu to enter any of five sub-systems. On exit, the patient list (including all visit histories) is serialised back to the binary file and all heap memory is freed.

```

main() [src/main.cpp]
|- loadPatientsFromBin() [src/PatientFile]
|- menuPatient() [src/PatientCRUD + src/PatientSearchSort]
|- menuQueue() -> addVisitHistory() [src/Queue + src/History]
|- medicineMenu() [src/Medicine]
|- menuHistory() [src/History]
|- menuStats() -> buildStats() -> exportReport()
| [src/Statistics + src/Reports]
'- savePatientsToBin() [src/PatientFile]
    -> patients.bin (project root)

```

Figure 1: High-level call graph

4. Detailed Feature Implementation

4.1. Patient Management - Dynamic Array (G1.2, G1.3)

Patient records are stored in `PatientList`, a heap-allocated dynamic array of `Patient` structs managed manually without STL containers.

```

1 struct PatientList {
2     Patient* data;
3     int count;
4     int capacity;
5 };
6
7 void resizePatientList(PatientList& list) {
8     int newCapacity = list.capacity * 2;
9     Patient* newData = new Patient[newCapacity];
10    for (int i = 0; i < list.count; i++)
11        newData[i] = list.data[i];
12    delete[] list.data;
13    list.data = newData;
14    list.capacity = newCapacity;
15 }

```

Listing 1: `PatientList` struct and resize logic (`Patient.h`, `PatientCRUD.cpp`)

When the array is full, capacity doubles - an amortised $O(1)$ insertion strategy. The `addPatient()` function also enforces unique IDs before insertion. `deletePatient()` first frees the patient's visit-history linked list to prevent memory leaks, then left-shifts the remaining elements.

Input validation (Task 11) is applied at entry: age is checked to be in range 1-150, name must be non-empty, and IDs are rejected if already present in the list.

4.2. Binary File Persistence (G2.3)

`src/PatientFile.cpp` implements custom binary serialisation without using `fwrite/fread` on raw structs (which would break for `std::string` fields). Instead, each string is length-prefixed:

```

1 static void writeString(ofstream& out, const string& str) {
2     size_t len = str.length();
3     out.write(reinterpret_cast<const char*>(&len), sizeof(len));
4     out.write(str.c_str(), len);
5 }
6
7 static string readString(istream& in) {
8     size_t len = 0;
9     in.read(reinterpret_cast<char*>(&len), sizeof(len));
10    char* buf = new char[len + 1];
11    in.read(buf, len);
12    buf[len] = '\0';
13    string str(buf);
14    delete[] buf;
15    return str;
16 }
```

Listing 2: Length-prefixed string serialisation (`PatientFile.cpp`)

Visit histories are serialised as a count followed by each node's three string fields. On load, `addVisitHistory()` reconstructs the linked list in the original chronological order. The save file `patients.bin` is written to and read from the project root (the working directory when `Clinic.exe` is launched).

4.3. Search and Sort - Quick Sort via Pointers (G1.4, G2.1)

Two Quick Sort variants are implemented in `src/PatientSearchSort.cpp`: ascending by name (A-Z) and descending by age (oldest first). A dedicated `swapPatientData()` function swaps every field individually (including the `historyHead` pointer) so visit history follows the correct patient after sorting.

```

1 static int partitionByName(PatientList& list, int low, int high)
2 {
3     string pivot = list.data[high].name;
4     int i = low - 1;
5     for (int j = low; j < high; j++) {
6         if (list.data[j].name <= pivot) {
7             i++;
8             swapPatientData(list.data[i], list.data[j]);
9         }
10    }
11    swapPatientData(list.data[i + 1], list.data[high]);
12    return i + 1;
13 }
```

Listing 3: Quick Sort partition by name (`PatientSearchSort.cpp`)

Name search is case-insensitive: both the stored name and the query are converted to lowercase before comparison using `find()`.

4.4. Examination Queue - FIFO Linked List (G3.2)

The queue is implemented as a singly linked list with separate `front` and `rear` pointers. Each `QueueNode` stores only the patient ID and name - not a pointer to the `Patient` - to avoid shallow-copy issues with `historyHead`.

```

1 void enqueue(Queue& q, const Patient& p) {
2     QueueNode* node = new QueueNode;
3     node->patientId   = p.id;
4     node->patientName = p.name;
5     node->next        = nullptr;
6     if (isEmpty(q)) { q.front = node; q.rear = node; }
7     else { q.rear->next = node; q.rear = node; }
8     q.count++;
9 }
10
11 bool dequeue(Queue& q, string& outId, string& outName) {
12     if (isEmpty(q)) return false;
13     QueueNode* tmp = q.front;
14     outId   = tmp->patientId;
15     outName = tmp->patientName;
16     q.front = q.front->next;
17     if (q.front == nullptr) q.rear = nullptr;
18     delete tmp;
19     q.count--;
20     return true;
21 }

```

Listing 4: Enqueue and dequeue (Queue.cpp)

When a patient is dequeued, `main()` prompts the user for consultation details (date, diagnosis, prescription) and calls `addVisitHistory()` to append a new node to that patient's linked list.

4.5. Visit History - Singly Linked List (G3.1)

Each `Patient` carries a `VisitNode*` head pointer. Nodes are always appended at the tail so that visits appear in chronological order.

```

1 void addVisitHistory(Patient& p,
2     const string& date, const string& diagnosis,
3     const string& prescription) {
4     VisitNode* newNode = createVisitNode(date, diagnosis,
5         prescription);
6     if (p.historyHead == nullptr) {
7         p.historyHead = newNode;
8         return;
9     }
10    VisitNode* cur = p.historyHead;
11    while (cur->next != nullptr)
12        cur = cur->next;
13    cur->next = newNode;
14 }

```

Listing 5: Append visit to tail (History.cpp)

`deleteVisitHistory()` traverses and frees every node, then sets the head to `nullptr`. It is called both from `deletePatient()` (to prevent memory leaks) and from the History sub-menu when the user clears a patient's record.

4.6. Medicine Storage - LIFO Stack (G3.2)

Medicine batches are managed with a linked-list stack in `src/Medicine.cpp`. `pushMedicine()` inserts at the head ($O(1)$); `popMedicine()` removes from the head ($O(1)$), implementing LIFO semantics so the most recently imported batch is dispensed first.

```

1 void pushMedicine(MedicineStack& s, const MedicineItem& item) {
2     MedicineNode* node = new MedicineNode;
3     node->data = item;
4     node->next = s.top;
5     s.top      = node;
6     s.count++;
7 }
8
9 bool popMedicine(MedicineStack& s, MedicineItem& outItem) {
10    if (isMedicineStackEmpty(s)) return false;
11    MedicineNode* tmp = s.top;
12    outItem = tmp->data;
13    s.top   = s.top->next;
14    delete tmp;
15    s.count--;
16    return true;
17 }

```

Listing 6: Push and pop (Medicine.cpp)

The sub-menu (`medicineMenu()`) uses the same arrow-key `runMenu()` navigation as the rest of the program, and validates that quantity is greater than zero before pushing.

4.7. Statistics - 2D Array (G1.4)

Visit counts are aggregated into a `int stats[31][12]` array indexed by `[day-1][month-1]`. `buildStats()` in `src/Statistics.cpp` traverses every patient's visit-history linked list and increments the corresponding cell after safe date parsing:

```

1 static bool parseDate(const string& date, int& day, int& month) {
2     if (date.length() < 10) return false;
3     if (date[2] != '/' || date[5] != '/') return false;
4     try {
5         day = stoi(date.substr(0, 2));
6         month = stoi(date.substr(3, 2));
7     } catch (...) { return false; }
8     return (day >= 1 && day <= 31 && month >= 1 && month <= 12);
9 }

```

Listing 7: Date parsing and accumulation (Statistics.cpp)

Only rows with at least one non-zero value are printed, keeping the table readable. The same array is passed to `exportReport()` for inclusion in the `.txt` file.

4.8. Report Export (G2.3)

`src/Reports.cpp` writes a three-section timestamped plain-text report:

1. **Section 1 - Patient Records:** formatted table of all patients.
2. **Section 2 - Visit Histories:** chronological visit list per patient.
3. **Section 3 - Visit Statistics:** day $\times$ month grid from the 2D array.

The timestamp is generated with `time()` and `strftime()`, so the report is self-dated at generation time.

4.9. Console UI (G4.1)

`src/Clinic.cpp` provides all terminal presentation logic using Windows API calls:

- `gotoxy()` / `setColor()` - direct cursor and colour control via `SetConsoleCursorPosition` and `SetConsoleTextAttribute`.
- `drawBox()` / `drawHeader()` - CP437 box-drawing characters for a bordered ASCII interface.
- `runMenu()` - arrow-key navigation using `_getch()`, returning the 1-based index of the selected option (0 on ESC). Used consistently across all menus including `medicineMenu()`.

The console is initialised with code page 437 (`SetConsoleOutputCP(437)`) to ensure box-drawing characters render correctly, and resized to 82 $\times$ 26 columns/lines via `system("mode con: ")`.

5. Testing

Table 3: Selected Test Cases

#	Test Scenario	Expected Result	Actual Result
1	Add patient with duplicate ID	Error message; record not inserted	Pass
2	Add patient with invalid age (0, 200, text)	Reprompt until valid age entered	Pass
3	Delete patient; reload from file	Record absent after restart	Pass
4	Sort by name, then reload from file	Sorted order persisted	Pass
5	Enqueue and dequeue patients	FIFO order maintained	Pass
6	Visit recorded after dequeue	History node appended; saved to file	Pass
7	Push/pop medicine batches	LIFO order; empty check on pop	Pass
8	Build stats with malformed dates	Malformed dates silently skipped	Pass
9	Export report with multiple patients	All sections present; timestamp correct	Pass
10	Search by name (mixed case)	Case-insensitive match returned	Pass

6. Team Contributions

Table 4: Work Distribution by Task

#	Task	Description	LO	Member
1	UI & Navigation	ASCII menu, arrow-key navigation, loading effects, box-drawing frames	G4.1	Huynh Thanh An
2	Patient CRUD	Add / View / Edit / Delete patient records using dynamic array	G1.2, G1.3	Ly Si Vi
3	Binary File Storage	Read/write binary file to persist patient list	G2.3	Ly Si Vi
4	Search & Sort	Search by ID/Name; sort list (A-Z, Age) using pointers	G1.4, G2.1	Ly Si Vi
5	Examination Queue	FIFO patient call system using Queue	G3.2	Luong Duy Phuoc
6	Visit History (Linked List)	Store each patient's visits in a singly linked list	G3.1	Luong Duy Phuoc
7	Medicine Storage (Stack)	Manage medicine import/export with LIFO (Stack)	G3.2	Huynh Thanh An
8	Statistics (2D Array)	Aggregate visit counts by Day $\times$ Month	G1.4	Tran Ngoc My Trinh
9	Report Export (.txt)	Export patient list & statistics to a text file	G2.3	Tran Ngoc My Trinh
10	Report & Documentation	Write project report, algorithm diagrams, bug fixes	–	Huynh Thanh An
11	Input Validation	Validate age (1-150), non-empty name, unique ID	G1.2, G1.3	Huynh Thanh An

Summary by Member

Member	Student ID	Responsibilities
Huynh Thanh An (Leader)	25127178	<code>src/Clinic.h/cpp</code> (UI engine, arrow-key menu, box drawing), <code>src/Medicine.h/cpp</code> (stack), <code>src/main.cpp</code> integration, input validation, report writing
Luong Duy Phuoc	24127509	<code>src/Queue.h/cpp</code> (FIFO examination queue), <code>src/History.h/cpp</code> (visit history linked list)
Ly Si Vi	24127587	<code>src/PatientCRUD.h/cpp</code> , <code>src/PatientFile.h/cpp</code> (binary I/O), <code>src/PatientSearchSort.h/cpp</code> (search & sort)
Tran Ngoc My Trinh	25127258	<code>src/Statistics.h/cpp</code> (2D array stats), <code>src/Reports.h/cpp</code> (text report export)

7. Conclusion

The Clinic Management System successfully meets all eleven tasks defined in the project proposal and satisfies learning outcomes G1 through G4. The implementation demonstrates practical use of:

- **Dynamic arrays** (auto-resizing `PatientList`) for G1.
- **Binary file I/O** with custom length-prefixed string serialisation for G2.
- **Linked-list queue, linked-list stack, and singly linked list** for G3.
- **Modular multi-file architecture** with clean header guards for G4.

Notable engineering decisions include using linked-list nodes in the queue and stack (rather than array-based implementations) to match the course's pointer-manipulation objectives, designing a custom binary serialiser that correctly handles `std::string` fields and nested visit-history data, and adopting a consistent arrow-key `runMenu()` interface across all sub-menus for a uniform user experience.

Future enhancements could include cross-platform terminal support (replacing Windows-specific APIs), a medicine-inventory persistence module, and a more sophisticated date-validation library.